

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Proyecto 01

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Identificación del Proyecto

Este documento corresponde al **Proyecto 01** de la materia *Modelado y Programación* de la Facultad de Ciencias, UNAM. El objetivo principal es implementar un sistema integral de ensamblaje y gestión de computadoras personalizadas para la empresa MonosChinos MX utilizando múltiples patrones de diseño que resuelvan los desafíos arquitectónicos del problema propuesto.

1.1. Descripción del Problema

MonosChinos MX es una empresa dedicada a la venta y ensamblaje de computadoras gamer en la Ciudad de México, operando inicialmente desde un modelo de negocio casero donde reciben pedidos vía redes sociales y ensamblan equipos con componentes Intel y NVIDIA. Debido al crecimiento exponencial de su popularidad en redes sociales por ofrecer computadoras con excelente relación calidad-precio, la empresa decide expandir sus operaciones estableciendo tres sucursales adicionales en Chihuahua, Jalisco y Yucatán, convirtiendo su sede original de la CDMX en una sucursal central dedicada exclusivamente a la línea de ensamblaje y distribución.

El sistema desarrollado permite construir configuraciones de computadoras mediante dos modalidades fundamentales: equipos prearmados seleccionados de un catálogo predefinido con configuraciones optimizadas para diferentes rangos de presupuesto (gama baja, media y alta), o equipos personalizados donde el cliente selecciona individualmente cada componente según sus preferencias específicas. Los componentes disponibles incluyen procesadores Intel y AMD, memorias RAM de Adata y Kingston, placas madre ASUS y MSI, discos duros mecánicos Western Digital y Seagate, discos de estado sólido Kingston, tarjetas gráficas NVIDIA, fuentes de alimentación EVGA, Corsair y XPG, y gabinetes NZXT y Yeyian.

El sistema implementa verificación automática de compatibilidad entre componentes, detectando problemas potenciales como procesadores AMD instalados en placas madre diseñadas exclusivamente para Intel, aplicando adaptadores cuando sea necesario para garantizar el funcionamiento correcto del equipo ensamblado. Adicionalmente, el sistema ofrece servicios opcionales de personalización de software que incluyen instalación de Windows 10/11, Microsoft Office 365, Adobe Photoshop, AutoCAD y configuración de terminal WSL, permitiendo que los clientes reciban equipos completamente configurados y listos para usar inmediatamente al conectarlos.

1.2. Arquitectura del Sistema

La solución implementa una arquitectura MVC (Modelo-Vista-Controlador) que separa claramente las responsabilidades de cada capa del sistema. El modelo gestiona la lógica de negocio incluyendo inventario de componentes, construcción de computadoras mediante builders especializados, verificación de compatibilidad mediante estrategias intercambiables, y persistencia de datos en base de datos SQLite. La vista proporciona dos interfaces distintas: una interfaz gráfica desarrollada en JavaFX con paneles especializados para selección de componentes, configuración de PC prearmadas, instalación de software y visualización de historial, y una interfaz de consola alternativa para ambientes sin capacidades gráficas. El controlador coordina la interacción entre modelo y vista mediante el patrón Facade que simplifica las operaciones complejas del sistema, delegando responsabilidades a los subsistemas especializados correspondientes.

Análisis de Implementación de Patrones

El sistema integra nueve patrones de diseño fundamentales que trabajan coordinadamente para proporcionar una arquitectura flexible, extensible y mantenible que resuelve eficazmente los requisitos del problema planteado mientras adhiere a principios SOLID que minimizan acoplamiento y maximizan cohesión entre componentes.

2.1. Patrón Builder

¿Por qué se eligió esta implementación?

El patrón Builder fue implementado para gestionar la construcción compleja de objetos *ComputadoraBase* que requieren ensamblaje secuencial de múltiples componentes interdependientes (CPU, RAM, MotherBoard, GPU, discos, fuente, gabinete), separando el algoritmo de construcción de la representación final del producto. Esta elección se fundamenta en la necesidad de construir dos tipos distintos de computadoras mediante procesos diferentes: equipos personalizados donde el usuario selecciona cada componente individualmente mediante la interfaz, y equipos prearmados que utilizan configuraciones predefinidas del inventario optimizadas para diferentes segmentos de mercado.

La implementación se estructura mediante la interfaz *PCBuilder* que define el contrato común para todos los constructores, estableciendo métodos para construir cada componente requerido (*construirCPU()*, *construirRAM()*, *construirMotherBoard()*, *construirGPU()*, *construirAlmacenamiento()*, *construirFuente()*, *construirGabinete()*), el método *reiniciar()* para comenzar una nueva construcción, y *obtenerResultado()* para recuperar el producto terminado. Esta abstracción permite que código cliente manipule builders de manera polimórfica sin conocimiento del tipo específico de computadora siendo construida.

La clase *PCPersonalizadoBuilder* implementa la construcción interactiva donde cada método solicita al usuario seleccionar el componente específico deseado de las opciones disponibles en el inventario, agregando el componente seleccionado a la computadora en construcción. Este builder mantiene flexibilidad completa permitiendo cualquier combinación de componentes que el usuario desee, delegando posteriormente la verificación de compatibilidad a las estrategias especializadas que detectan y resuelven conflictos potenciales.

Paralelamente, la clase *PCPrearmadoBuilder* implementa la construcción automática basada en configuraciones predefinidas almacenadas en el inventario (gama baja, media, alta), donde cada método de construcción recupera el componente apropiado de la configuración seleccionada sin intervención del usuario. Esta implementación proporciona experiencia simplificada para clientes que prefieren configuraciones probadas y optimizadas sin navegar opciones individuales de componentes.

La clase *DirectorPC* coordina el proceso de construcción mediante métodos que invocan secuencialmente todos los pasos del builder en el orden correcto, garantizando que ningún componente esencial sea omitido. El director proporciona métodos especializados *construirPCPersonalizada()* y *construirPCPrearmada()* que adaptan la secuencia de construcción según el tipo de equipo deseado, encapsulando el conocimiento de cuándo invocar cada método del builder.

La integración del patrón en el sistema permite agregar nuevos tipos de builders sin modificar código existente, facilitando extensiones futuras como builders para equipos orientados a productividad, servidores, o estaciones de trabajo profesionales. La separación clara entre construcción y representación maximiza reusabilidad del código y simplifica pruebas unitarias al permitir verificación independiente de cada componente del builder.

2.2. Patrón Adapter

¿Por qué se eligió esta implementación?

El patrón Adapter fue implementado para integrar procesadores AMD dentro del sistema originalmente diseñado para componentes Intel, permitiendo que CPUs AMD participen transparentemente en todas las operaciones del sistema sin modificar las interfaces existentes de *ComponentePC* que esperan características específicas de procesadores Intel. Esta elección responde al requisito del negocio de expandir el mercado soportando procesadores AMD después de recibir críticas por ofrecer exclusivamente equipos con arquitectura Intel, manteniendo compatibilidad hacia atrás con el código legacy mientras se incorpora nueva funcionalidad.

La implementación se materializa mediante la clase *CPUAMDAdapter* que implementa la interfaz *ComponentePC* estableciendo compatibilidad estructural con el resto del sistema, mientras mantiene internamente una instancia de CPU que representa el procesador AMD real con sus atributos específicos de arquitectura x86-64 AMD64, número de núcleos y características técnicas distintivas. Esta relación de composición permite que el adaptador delegue operaciones al objeto CPU encapsulado mientras traduce llamadas

entre interfaces potencialmente incompatibles.

Los métodos de la interfaz `ComponentePC` como `obtenerNombre()`, `obtenerPrecio()`, `obtenerMarca()`, `obtenerTipo()` y `mostrarDetalles()` son implementados mediante delegación directa al CPU AMD encapsulado, aprovechando que estas operaciones tienen correspondencia natural entre ambas arquitecturas de procesadores. El adaptador actúa como proxy transparente que redirige invocaciones sin transformación adicional cuando la semántica de las operaciones permanece consistente entre Intel y AMD.

El método adicional `adaptar()` proporciona capacidad de introspección permitiendo que código que conoce específicamente el adaptador pueda obtener información sobre el proceso de adaptación realizado, retornando mensaje descriptivo que indica el CPU AMD específico siendo adaptado como `ComponentePC`. Esta funcionalidad facilita depuración y logging detallado del comportamiento del sistema cuando trabaja con componentes adaptados.

La integración del patrón en el sistema permite que las estrategias de compatibilidad como `CompatibilidadMixta` detecten procesadores AMD adaptados y apliquen verificaciones especiales para garantizar que placas madre Intel sean reemplazadas por adaptadores apropiados cuando sea necesario. El sistema verifica automáticamente la arquitectura del procesador mediante análisis del nombre o marca, aplicando transformaciones que garantizan funcionamiento correcto del equipo ensamblado independientemente de la combinación de componentes seleccionados.

2.3. Patrón Decorator

¿Por qué se eligió esta implementación?

El patrón Decorator fue implementado para agregar dinámicamente capacidades de software a objetos `ComputadoraBase` sin modificar su estructura fundamental, permitiendo que computadoras sean extendidas con múltiples aplicaciones opcionales (Windows, Office, Photoshop, AutoCAD, WSL) mediante composición recursiva que mantiene la interfaz uniforme. Esta elección responde al requisito de ofrecer servicios opcionales de personalización de software donde clientes pueden seleccionar selectivamente qué aplicaciones desean preinstaladas, con la restricción de que cada software puede agregarse únicamente una vez por computadora.

La implementación se estructura mediante la clase abstracta `SoftwareDecorator` que implementa `ComputadoraBase` estableciendo compatibilidad con computadoras no decoradas, mientras mantiene referencia al objeto `ComputadoraBase` siendo decorado junto con atributos `nombreSoftware` y `precioSoftware` que almacenan información del software específico agregado. Esta clase proporciona implementaciones por defecto de métodos como `obtenerPrecioTotal()` que suma recursivamente precios mediante delegación a la computadora decorada, y `obtenerComponentes()` que delega directamente sin modificación.

Las clases concretas `WindowsDecorator`, `OfficeDecorator`, `PhotoshopDecorator`, `AutoCADDecorator` y `WSLDecorator` extienden el decorador abstracto proporcionando implementaciones específicas del método `obtenerDescripcion()` que agrega información del software particular a la descripción de la computadora decorada. Cada decorador concreto inicializa el nombre y precio apropiados del software mediante invocación al constructor de la clase base, encapsulando detalles específicos de cada aplicación.

El método `tieneSoftware()` implementa búsqueda recursiva en la cadena de decoradores mediante comparación case-insensitive del nombre del software buscado con el software del decorador actual, delegando recursivamente a la computadora decorada cuando no coincide. Esta implementación permite que el sistema verifique existencia de software específico independientemente de la profundidad de anidación o posición en la cadena de decoración, habilitando validación que previene instalación duplicada del mismo software.

La composición recursiva permite crear cadenas arbitrariamente largas de decoradores donde cada uno agrega su funcionalidad específica, calculando automáticamente precio total acumulado y generando descripciones que incluyen todos los softwares instalados. El sistema aplica decoradores secuencialmente

según selección del usuario, verificando antes de cada adición que el software no haya sido previamente instalado mediante invocación a `tieneSoftware()`.

2.4. Patrón Factory (Abstract Factory)

¿Por qué se eligió esta implementación?

El patrón Abstract Factory fue implementado mediante las clases `IntelFactory` y `AMDFactory` que encapsulan creación de familias de componentes relacionados específicos de cada fabricante, proporcionando interfaz unificada para instanciar procesadores, placas madre y otros componentes que mantienen coherencia arquitectónica dentro de cada familia. Esta elección se fundamenta en la necesidad de garantizar que componentes creados por cada factoría sean inherentemente compatibles entre sí, reduciendo complejidad de verificaciones de compatibilidad y facilitando extensión futura con nuevos fabricantes.

La clase `IntelFactory` implementa métodos de creación que instancian componentes Intel específicos como procesadores Core i3, i5, i7 e i9 con arquitectura x86-64, junto con placas madre ASUS y MSI diseñadas para socket LGA1700 compatible con procesadores Intel de 13va generación. Esta factoría garantiza que todos los componentes creados mantienen consistencia arquitectónica necesaria para ensamblaje exitoso sin conflictos de compatibilidad entre procesador y placa madre.

Paralelamente, `AMDFactory` proporciona creación de procesadores AMD Ryzen 5, 7 y 9 con arquitectura x86-64 AMD64, encapsulando detalles de instanciación específicos de componentes AMD. La factoría permite que el sistema agregue soporte para nuevas familias de procesadores AMD sin modificar código cliente que depende de la interfaz abstracta de creación, maximizando extensibilidad y facilitando incorporación de componentes futuros conforme el inventario evoluciona.

La integración del patrón con el sistema de inventario permite que `Inventario` utilice ambas factorías durante inicialización para poblar el catálogo con componentes Intel y AMD, delegando responsabilidad de creación a objetos especializados que conocen detalles técnicos de cada familia. Esta separación de responsabilidades simplifica mantenimiento del inventario y facilita adición de nuevos fabricantes como ASUS, MSI u otros proveedores de componentes mediante implementación de nuevas factorías concretas.

2.5. Patrón Strategy

¿Por qué se eligió esta implementación?

El patrón Strategy fue implementado para encapsular algoritmos de verificación de compatibilidad entre componentes en clases intercambiables independientes (`CompatibilidadIntel`, `CompatibilidadAMD`, `CompatibilidadMixta`), permitiendo que el sistema seleccione dinámicamente la estrategia apropiada según la combinación de componentes detectada sin lógica condicional compleja en código cliente. Esta elección responde a la necesidad de verificar compatibilidad mediante algoritmos distintos dependiendo de si el equipo utiliza exclusivamente componentes Intel, exclusivamente AMD, o mezcla componentes de ambos fabricantes requiriendo adaptaciones especiales.

La interfaz `EstrategiaCompatibilidad` define el contrato común mediante el método `verificarCompatibilidad()` que recibe lista de componentes y retorna objeto `ResultadoCompatibilidad` encapsulando si los componentes son compatibles, mensaje descriptivo del resultado, y lista de advertencias detectadas durante verificación. Esta abstracción permite que nuevas estrategias sean agregadas sin modificar código que las utiliza, facilitando extensión futura con verificaciones adicionales como consumo eléctrico, compatibilidad térmica, o restricciones físicas de espacio.

La clase `CompatibilidadIntel` implementa verificación especializada para equipos con procesadores Intel, validando que la placa madre sea compatible con socket LGA1700 y arquitectura x86-64 esperada por procesadores Intel de 13va generación. Esta estrategia retorna resultados exitosos cuando todos los componentes mantienen coherencia arquitectónica Intel, o genera advertencias cuando detecta inconsistencias que podrían causar problemas durante ensamblaje.

Paralelamente, `CompatibilidadAMD` verifica equipos con procesadores AMD Ryzen, validando compatibilidad con placas madre diseñadas para arquitectura AMD64 y sockets apropiados para las familias Ryzen 5000 y 7000. Esta estrategia reconoce que el inventario actual carece de placas madre AMD nativas, generando advertencias cuando detecta esta situación para informar al usuario sobre limitaciones del catálogo disponible.

La estrategia `CompatibilidadMixta` implementa lógica sofisticada que detecta procesadores AMD instalados en placas madre Intel, aplicando automáticamente `CPUAMDAdapter` para garantizar interoperabilidad entre arquitecturas incompatibles. Esta estrategia genera advertencias explicando la adaptación realizada y recomendando adquisición futura de placas madre AMD nativas para optimizar rendimiento, mientras permite que el usuario proceda con la configuración adaptada si así lo desea.

El sistema selecciona la estrategia apropiada mediante análisis de los componentes presentes en la computadora siendo verificada, delegando responsabilidad de verificación al objeto estrategia correspondiente que implementa algoritmo especializado. Esta arquitectura permite que lógica de compatibilidad evolucione independientemente del resto del sistema, facilitando refinamiento de reglas conforme se incorporan nuevos componentes al inventario.

2.6. Patrón Composite

¿Por qué se eligió esta implementación?

El patrón Composite fue implementado para gestionar componentes que pueden ser tratados tanto individualmente como en agrupaciones mediante jerarquías parte-todo que unifican componentes atómicos y compuestos bajo interfaz común `ComponentePC`. Esta elección responde al requisito de permitir instalación de múltiples módulos de memoria RAM y múltiples discos de almacenamiento, creando objetos `JuegoRAMs` y `JuegoDiscos` que agrupan componentes individuales mientras se comportan como componentes únicos para el resto del sistema.

La interfaz `ComponentePC` establece operaciones fundamentales como `obtenerNombre()`, `obtenerPrecio()`, `obtenerMarca()`, `obtenerTipo()` y `mostrarDetalles()` que deben ser implementadas tanto por componentes atómicos (hojas) como por componentes compuestos (composites), garantizando tratamiento uniforme independientemente de la complejidad estructural subyacente. Esta abstracción permite que código cliente manipule configuraciones simples y complejas mediante referencias polimórficas sin conocimiento de la composición interna.

La clase `ComponenteHoja` implementa componentes atómicos individuales como CPU, GPU, MotherBoard, FuenteAlimentacion y Gabinete que no contienen subcomponentes, proporcionando implementaciones directas de todos los métodos heredados que retornan valores almacenados sin cálculos recursivos. Las clases especializadas CPU, RAM, Disco, GPU, MotherBoard, FuenteAlimentacion y Gabinete extienden `ComponenteHoja` agregando atributos específicos de cada tipo de componente como número de núcleos en CPUs, capacidad en RAMs y discos, o tipo de memoria en GPUs.

Las clases `JuegoRAMs` y `JuegoDiscos` implementan el componente compuesto del patrón mediante gestión de colecciones internas de componentes individuales, proporcionando métodos `agregarComponente()` y `removerComponente()` para manipular dinámicamente la composición. El método `obtenerPrecio()` implementa cálculo recursivo que suma precios de todos los componentes contenidos mediante programación funcional con streams, mientras que `obtenerNombre()` y `mostrarDetalles()` generan descripciones que incluyen información de todos los subcomponentes.

La integración del patrón en el sistema permite que builders agreguen múltiples RAMs o discos creando automáticamente objetos `JuegoRAMs` o `JuegoDiscos` que encapsulan la colección, simplificando gestión posterior de configuraciones que contienen componentes múltiples. El sistema trata juegos de componentes transparentemente como componentes individuales durante cálculos de precio total, generación de tickets y persistencia en base de datos, demostrando la potencia del patrón para ocultar complejidad estructural detrás de interfaces uniformes.

2.7. Patrón Facade

¿Por qué se eligió esta implementación?

El patrón Facade fue implementado mediante la clase `SistemaEnsamblajeFacade` que proporciona interfaz simplificada para coordinar operaciones complejas del sistema de ensamblaje, encapsulando interacciones entre múltiples subsistemas como inventario, builders, estrategias de compatibilidad, generación de tickets y persistencia. Esta elección se fundamenta en la necesidad de reducir acoplamiento entre controladores de vista y subsistemas de modelo, proporcionando punto de entrada unificado que simplifica uso del sistema y oculta complejidades arquitectónicas internas.

La fachada encapsula instanciación y coordinación del `Inventario` singleton que gestiona el catálogo completo de componentes disponibles, `DirectorPC` que coordina builders para construcción de computadoras, implementaciones concretas de `PCBuilder` para equipos personalizados y prearmados, y `ServicioPersistencia` que gestiona almacenamiento en base de datos SQLite. Esta centralización de dependencias permite que controladores invoquen operaciones de alto nivel sin conocer detalles de implementación subyacentes.

Los métodos de la fachada como `obtenerComponentesPorTipo()`, `obtenerConfiguracionesPrearmadas()`, `construirPCPersonalizada()`, `construirPCPrearmada()`, `verificarCompatibilidad()`, `generarTicket()` y `guardarTicket()` proporcionan operaciones cohesivas de alto nivel que coordinan múltiples subsistemas mediante secuencias de invocaciones apropiadas. Cada método encapsula flujo de control complejo delegando responsabilidades a objetos especializados que implementan funcionalidad específica.

La implementación de métodos como `construirPCPersonalizada()` demuestra coordinación compleja donde la fachada instancia `PCPersonalizadoBuilder`, configura `DirectorPC` con el builder, invoca construcción paso a paso, recupera resultado, aplica verificación de compatibilidad mediante estrategia apropiada, y retorna computadora construida junto con resultado de compatibilidad. Esta orquestación oculta complejidad al controlador que simplemente invoca un método unificado recibiendo resultado completo.

La integración del patrón facilita testing mediante inyección de implementaciones mock de la fachada durante pruebas unitarias de controladores, permitiendo verificación aislada de lógica de coordinación sin dependencias reales de inventario o persistencia. La fachada también simplifica evolución del sistema al centralizar cambios en un único punto cuando subsistemas internos requieren refactorización, protegiendo controladores de impacto de modificaciones arquitectónicas internas.

2.8. Patrón Singleton

¿Por qué se eligió esta implementación?

El patrón Singleton fue implementado en la clase `Inventario` para garantizar existencia de una única instancia global del catálogo de componentes compartida por todo el sistema, evitando duplicación innecesaria de datos y garantizando consistencia del estado del inventario independientemente del punto de acceso. Esta elección se fundamenta en la naturaleza centralizada del inventario donde todos los componentes disponibles deben ser gestionados coherentemente sin posibilidad de catálogos múltiples desincronizados.

La implementación utiliza el enfoque clásico de Singleton con constructor privado que previene instanciación externa mediante operador `new`, variable estática privada `instancia` que almacena la única instancia del inventario, y método estático público `getInstance()` que controla creación y acceso a la instancia mediante inicialización perezosa que crea el objeto únicamente cuando es solicitado por primera vez. El método utiliza programación funcional con `Optional.ofNullable()` y `orElseGet()` para expresar elegantemente la lógica de creación condicional.

Durante inicialización, el inventario invoca `inicializarCatalogo()` que puebla el catálogo mediante invocación de métodos especializados `crearListaCPUs()`, `crearListaRAMs()`, `crearListaMotherBoards()`,

`crearListaDiscos()`, `crearListaGPUs()`, `crearListaFuentes()` y `crearListaGabinetes()` que instancian todos los componentes disponibles con información completa de nombres, precios, marcas y especificaciones técnicas. Esta inicialización se ejecuta únicamente una vez durante la primera invocación a `getInstance()`, optimizando uso de memoria y tiempo de inicio.

El inventario proporciona métodos de consulta como `obtenerComponentesPorTipo()` que retorna copia defensiva de la lista de componentes del tipo especificado para prevenir modificaciones externas de colecciones internas, y `obtenerComponente()` que busca componente específico por tipo y nombre mediante streams funcionales con `filter()` y `findFirst()`. Adicionalmente, `obtenerConfiguracionesPrearmadas()` proporciona acceso a configuraciones predefinidas de gama baja, media y alta que combinan componentes seleccionados para diferentes segmentos de mercado.

La integración del patrón garantiza que todas las partes del sistema (builders, fachada, controladores, vistas) acceden al mismo catálogo consistente de componentes sin duplicación de datos o desincronización de estado. El Singleton facilita también modificaciones dinámicas del inventario mediante método `agregarComponente()` que permite incorporar nuevos productos durante ejecución, reflejando inmediatamente los cambios en todas las partes del sistema que consultan el inventario.

2.9. Patrón Iterator

¿Por qué se eligió esta implementación?

El patrón Iterator fue implementado mediante la clase `IteratorComponentePC` que proporciona mecanismo estándar para recorrer secuencialmente colecciones de componentes sin exponer representación interna subyacente, permitiendo navegación uniforme independientemente de si la colección es lista, conjunto u otra estructura de datos. Esta elección responde a la necesidad de procesar componentes de computadoras durante operaciones como generación de tickets, cálculo de precios totales y visualización de detalles, manteniendo encapsulación de implementación de colección que podría evolucionar sin impactar código cliente.

La implementación encapsula lista interna de componentes junto con índice de posición actual, proporcionando métodos `hasNext()` que verifica existencia de elementos adicionales comparando índice con tamaño de lista, y `next()` que retorna componente en posición actual incrementando posteriormente el índice. Esta interfaz minimalista permite que loops utilicen el iterador mediante patrón idiomático `while (iterator.hasNext()) { ... iterator.next() ... }` sin conocimiento de detalles de almacenamiento subyacente.

El iterador es utilizado durante generación de tickets donde el sistema recorre componentes de computadora construida extrayendo información relevante como nombre, precio y detalles técnicos para formatear ticket detallado que documenta configuración completa del equipo junto con precio total y advertencias de compatibilidad. Esta navegación secuencial mediante iterador abstrae complejidad de acceso a componentes individuales manteniendo flexibilidad de implementación subyacente.

Adicionalmente, el iterador facilita implementación de operaciones funcionales como filtrado, mapeo y reducción sobre colecciones de componentes, proporcionando base para transformaciones declarativas que expresan qué procesar sin especificar cómo iterar. Esta capacidad habilita implementaciones elegantes de operaciones como cálculo de precio total mediante `reduce` que suma precios iterando componentes, o generación de descripciones mediante `map` que extrae nombres concatenando resultados.

Catálogo de Componentes

El sistema gestiona un inventario completo de componentes organizados por categorías con diversidad de marcas, especificaciones técnicas y rangos de precios que permiten ensamblar equipos desde configuraciones económicas de gama baja hasta estaciones de trabajo de alto rendimiento para entusiastas y profesionales.

3.1. Procesadores (CPU)

El catálogo incluye procesadores Intel Core de 13va generación (i3-13100, i5-13600K, i7-13700K, i9-13900K) con arquitectura x86-64 y procesadores AMD Ryzen serie 5000 y 7000 (Ryzen 5 5600G, Ryzen 5 7600X, Ryzen 7 7700X, Ryzen 9 7950X3D) con arquitectura x86-64 AMD64. Los procesadores disponibles cubren rangos desde 4 hasta 24 núcleos físicos con precios entre **\$7,999.00 y \$34,999.00 pesos mexicanos**, proporcionando opciones para diferentes niveles de exigencia computacional desde navegación básica hasta renderizado profesional y gaming competitivo de alta frecuencia.

3.2. Memoria RAM

Se ofrecen módulos de memoria DDR4 de las marcas Adata y Kingston en capacidades de 8GB, 16GB y 32GB por módulo, permitiendo configuraciones desde 8GB totales para uso básico hasta 64GB o más para estaciones de trabajo mediante instalación de múltiples módulos. Los precios oscilan entre **\$599.00 y \$2,599.00 pesos mexicanos** por módulo, con la posibilidad de instalar múltiples unidades para maximizar capacidad de memoria según requisitos de aplicaciones utilizadas.

3.3. Placas Madre (MotherBoard)

El inventario incluye placas madre de gama media y alta de ASUS (ROG Maximus Z790 Hero, TUF Gaming B760-Plus WIFI D4) y MSI (MEG Z790 Godlike, MAG B760 Tomahawk WIFI DDR4) diseñadas para procesadores Intel con socket LGA1700 y chipsets Z790 o B760. Los precios varían entre **\$7,999.00 y \$29,999.00 pesos mexicanos**, reflejando diferencias en características como capacidades de overclocking, conectividad WiFi integrada, número de slots de expansión y soporte para almacenamiento NVMe de alta velocidad.

3.4. Almacenamiento (Discos)

Se dispone de discos duros mecánicos HDD de Western Digital Blue y Seagate Barracuda en capacidades de 500GB a 2TB con precios entre **\$899.00 y \$1,999.00 pesos mexicanos**, apropiados para almacenamiento masivo de bajo costo. Complementariamente, discos de estado sólido SSD Kingston A400 ofrecen capacidades de 500GB a 4TB con precios entre **\$1,299.00 y \$8,299.00 pesos mexicanos**, proporcionando velocidades de lectura/escritura significativamente superiores para sistema operativo y aplicaciones frecuentemente utilizadas. El sistema permite instalar múltiples unidades combinando HDDs para almacenamiento y SSDs para rendimiento.

3.5. Tarjetas Gráficas (GPU)

El catálogo incluye tarjetas gráficas NVIDIA desde la GTX 1660 con memoria GDDR5 de 6GB hasta la RTX 4090 con memoria GDDR6X de 24GB, cubriendo rangos de precios entre **\$4,999.00 y \$39,999.00 pesos mexicanos**. Las opciones intermedias incluyen RTX 3060, RTX 4070 y RTX 4080 que proporcionan diferentes niveles de rendimiento para gaming en resoluciones 1080p, 1440p y 4K, así como capacidades de ray tracing en tiempo real y aceleración de inteligencia artificial mediante núcleos tensor especializados.

3.6. Fuentes de Alimentación

Se ofrecen fuentes de alimentación de EVGA, Corsair y XPG con potencias desde 500W hasta 1500W y certificaciones de eficiencia 80 PLUS Bronze, Gold, Platinum y Titanium. Los precios oscilan entre **\$1,299.00 y \$5,299.00 pesos mexicanos**, permitiendo selección apropiada según consumo total del sistema calculado a partir de especificaciones de CPU, GPU y componentes adicionales instalados.

3.7. Gabinetes

El inventario incluye gabinetes ATX de NZXT (H6 Flow ATX) y Yeyian (Lancer ATX) con precios entre **\$1,899.00 y \$2,299.00 pesos mexicanos**, proporcionando factor de forma compatible con las placas madre disponibles y capacidad para instalar sistemas de refrigeración estándar que garanticen temperaturas operativas apropiadas.

Dependencias Externas del Proyecto

El proyecto integra múltiples bibliotecas externas especializadas que proporcionan funcionalidades avanzadas para desarrollo de interfaz gráfica, persistencia de datos, programación funcional y testing automatizado, declaradas en el archivo `pom.xml` y gestionadas automáticamente por Maven durante compilación y empaquetado.

4.1. JavaFX 17.0.13

Framework oficial de Oracle para desarrollo de interfaces gráficas de usuario modernas en aplicaciones Java, proporcionando componentes visuales avanzados (botones, tablas, listas, paneles), sistema de layouts responsivos que adaptan diseño a diferentes tamaños de ventana, motor de renderizado acelerado por hardware para animaciones fluidas, y soporte para hojas de estilo CSS que permiten personalización completa de apariencia visual. El proyecto utiliza los módulos `javafx-controls` para componentes básicos de interfaz, `javafx-fxml` para declaración de interfaces mediante XML, y `javafx-graphics` para operaciones de renderizado y gestión de escenas gráficas.

4.2. SQLite JDBC 3.47.1.0

Driver JDBC que permite comunicación con bases de datos SQLite embebidas directamente en la aplicación sin requerir servidor de base de datos separado, ideal para aplicaciones de escritorio que necesitan persistencia sin complejidad de instalación o configuración de infraestructura externa. El sistema utiliza SQLite para almacenar historial de tickets generados, permitiendo que usuarios consulten pedidos previos y rastreen configuraciones de equipos ensamblados anteriormente. La base de datos se almacena en archivo único fácilmente respaldable que contiene esquema completo y todos los datos persistidos.

4.3. Gson 2.11.0

Biblioteca de Google para serialización y deserialización automática de objetos Java hacia formato JSON (JavaScript Object Notation) ampliamente utilizado para intercambio de datos estructurados. Gson analiza introspectivamente estructuras de objetos generando representaciones JSON equivalentes, y reconstruye objetos Java desde JSON mediante reflexión que mapea campos automáticamente. El proyecto utiliza Gson para persistir configuraciones complejas de computadoras con sus componentes anidados en formato legible que facilita depuración e intercambio de datos con sistemas externos.

4.4. ControlsFX 11.2.1

Biblioteca comunitaria que extiende JavaFX con componentes avanzados adicionales no incluidos en distribución estándar, proporcionando controles como notificaciones toast, diálogos personalizados, campos de texto con autocompletado, selectores de color avanzados, barras de progreso mejoradas y validación automática de formularios. El proyecto aprovecha estos componentes para mejorar experiencia de usuario mediante interfaces más interactivas y profesionales que simplifican operaciones comunes como selección de componentes con búsqueda incremental.

4.5. Ikonli (JavaFX + FontAwesome5)

Sistema de iconos vectoriales que integra FontAwesome 5 en aplicaciones JavaFX, proporcionando miles de iconos escalables profesionales que mantienen calidad visual independientemente del tamaño de renderizado. Los módulos `ikonli-javafx` y `ikonli-fontawesome5-pack` permiten incorporar iconos mediante códigos simples que referencian glyphs vectoriales, facilitando creación de interfaces visualmente atractivas con iconografía consistente y reconocible que mejora usabilidad mediante señales visuales claras de funcionalidad de cada elemento.

4.6. Vavr 0.10.4

Biblioteca de programación funcional que extiende Java con estructuras de datos inmutables y persistentes (listas, conjuntos, mapas), tipos algebraicos como `Option`, `Either` y `Try` para manejo elegante de errores sin excepciones, y operaciones funcionales avanzadas como `pattern matching` y composición de funciones. El proyecto utiliza Vavr para implementar transformaciones de datos mediante pipelines funcionales

que expresan declarativamente qué computar sin especificar imperativamente cómo iterar estructuras, mejorando legibilidad y reduciendo errores mediante eliminación de estado mutable.

4.7. Apache Commons Lang3 3.14.0

Biblioteca de utilidades que proporciona implementaciones robustas de operaciones comunes sobre strings, arrays, números, fechas y objetos que complementan biblioteca estándar de Java. Incluye funciones para manipulación avanzada de texto, validación de entradas, generación de hashes, comparación profunda de objetos y manejo seguro de valores nulos. El proyecto utiliza estas utilidades para simplificar operaciones repetitivas manteniendo código conciso y evitando reimplementación de funcionalidad ya probada extensivamente por la comunidad.

4.8. JUnit 5.11.3

Framework líder de la industria para desarrollo de pruebas unitarias automatizadas en Java, proporcionando anotaciones para declarar métodos de prueba, assertions para verificar condiciones esperadas, fixtures para configurar estado previo a pruebas, y runners para ejecutar suites completas de tests. El proyecto incluye `junit-jupiter-api` con API principal, `junit-jupiter-engine` con motor de ejecución, y `junit-jupiter-params` para pruebas parametrizadas que verifican múltiples casos con diferentes entradas. El sistema implementa **534 pruebas unitarias** que verifican exhaustivamente comportamiento de componentes individuales, builders, estrategias, decoradores y fachada, garantizando calidad mediante validación automatizada de requisitos funcionales.

Requisitos del Sistema

El sistema requiere configuración específica de software y herramientas de desarrollo para compilación, ejecución y despliegue exitosos en ambientes de desarrollo y producción.

5.1. Java Development Kit 17

El proyecto requiere obligatoriamente **JDK 17** (Java Development Kit versión 17) como versión de soporte a largo plazo que proporciona estabilidad, rendimiento optimizado y características modernas del lenguaje. Esta versión introduce mejoras significativas en garbage collection, optimizaciones de máquina virtual que aceleran ejecución de código polimórfico intensivo como el utilizado en patrones de diseño, y APIs mejoradas para programación funcional mediante streams y optional. La compatibilidad del código se verifica mediante configuración explícita en `pom.xml` que establece `maven.compiler.source` y `maven.compiler.target` a versión 17.

5.2. Apache Maven 3.x

Maven es la herramienta de automatización de compilación y gestión de dependencias utilizada por el proyecto, encargada de descargar automáticamente bibliotecas externas desde repositorios centrales, compilar código fuente en bytecode optimizado, ejecutar suites completas de pruebas unitarias, empaquetar aplicación en archivos JAR ejecutables con todas las dependencias necesarias, y generar documentación Javadoc. Se requiere **Maven 3.x** instalado y configurado en PATH del sistema para ejecutar comandos de compilación y gestión del ciclo de vida del proyecto.

5.3. Docker

El despliegue del sistema requiere **Docker** como tecnología de contenedores obligatoria según especificaciones del proyecto, permitiendo empaquetar aplicación completa con todas sus dependencias en imagen portable que ejecuta consistentemente en cualquier ambiente sin variaciones por diferencias de sistema operativo o configuración. Docker garantiza que aplicación funcione idénticamente en máquinas de desarrollo, servidores de pruebas y ambientes de producción, eliminando problemas clásicos de "funciona en mi máquina" mediante aislamiento completo de dependencias. El usuario debe tener Docker instalado y en ejecución, con permisos apropiados para construir imágenes y ejecutar contenedores sin requerir privilegios de superusuario.

Compilación y Ejecución con Maven

Maven proporciona comandos estandarizados para gestionar el ciclo de vida completo del proyecto desde compilación inicial hasta empaquetado y ejecución, automatizando tareas repetitivas mediante convenciones sobre configuración.

6.1. Comandos Principales de Maven

Limpieza de Compilaciones Previas

```
mvn clean
```

Elimina directorio `target/` que contiene bytecode compilado y artefactos generados en ejecuciones previas, garantizando compilación limpia sin conflictos por archivos obsoletos remanentes de versiones anteriores del código.

Compilación del Proyecto

```
mvn compile
```

Compila todo el código fuente Java ubicado en `src/main/java/` generando bytecode optimizado en `target/classes/`, resolviendo automáticamente dependencias declaradas en `pom.xml` descargando bibliotecas necesarias desde repositorio Maven Central cuando no estén disponibles en cache local.

Ejecución de Pruebas Unitarias

```
mvn test
```

Ejecuta las **534 pruebas unitarias** implementadas en `src/test/java/` utilizando JUnit 5, verificando exhaustivamente comportamiento correcto de componentes, builders, estrategias, decoradores y fachada mediante assertions que validan postcondiciones esperadas. El comando compila automáticamente código de pruebas si no existe bytecode actualizado, genera reportes detallados con resultados de cada test indicando cuáles pasaron exitosamente y cuáles fallaron con trazas de error, y termina con código de salida distinto de cero si cualquier prueba falla.

Empaquetado en JAR Ejecutable

```
mvn package
```

Genera archivo JAR ejecutable en `target/sistema-ensamblaje-pc-1.0.0.jar` que contiene bytecode compilado y manifiesto configurado con clase principal para ejecución directa. Adicionalmente copia todas las dependencias externas a `target/lib/` mediante plugin `maven-dependency-plugin`, permitiendo distribución completa de aplicación con todas las bibliotecas necesarias. Este comando ejecuta automáticamente fases previas (`compile`, `test`) garantizando que `package` se genera únicamente si todas las pruebas pasan exitosamente.

Ejecución de Aplicación de Consola

```
mvn exec:java
```

Ejecuta la clase principal `Main.java` que proporciona interfaz de consola basada en texto para interacción con el sistema mediante menús de selección de opciones y entrada por teclado. Esta modalidad funciona en cualquier ambiente con terminal sin requerir capacidades gráficas, apropiada para servidores o ambientes de desarrollo remoto.

Ejecución de Aplicación GUI JavaFX

```
mvn javafx:run
```

Ejecuta la clase principal MainJavaFX que inicializa aplicación JavaFX con interfaz gráfica completa incluyendo ventana principal, paneles especializados para selección de componentes, configuración de PC prearmadas, instalación de software y visualización de historial. Esta modalidad requiere ambiente con servidor X Window (Linux), Quartz (macOS) o GDI (Windows) para renderizado de componentes gráficos.

Despliegue con Docker (Obligatorio)

El sistema debe desplegarse obligatoriamente mediante tecnología de contenedores Docker que encapsula aplicación completa con todas sus dependencias en imagen portable ejecutable consistentemente en cualquier ambiente que soporte Docker.

7.1. Construcción de la Imagen Docker

Construir Imagen

```
docker build -t monoschinos-mx .
```

Construye imagen Docker con etiqueta monoschinos-mx ejecutando instrucciones del Dockerfile que configuran imagen base openjdk:17-slim con JRE optimizado, instalan Maven para compilación, copian código fuente completo incluyendo pom.xml con dependencias, ejecutan mvn package dentro del contenedor para generar JAR con todas las bibliotecas, y configuran comando de inicio por defecto que ejecuta aplicación JavaFX. La construcción completa requiere aproximadamente 5-10 minutos descargando dependencias y compilando, generando imagen final de aproximadamente 955MB que contiene entorno de ejecución completo autocontenido.

7.2. Verificación de Imagen

Listar Imágenes Docker

```
docker images monoschinos-mx
```

Lista imagen construida mostrando información de repositorio, etiqueta, ID único de imagen, fecha de creación y tamaño total. Permite verificar que construcción completó exitosamente y la imagen está disponible localmente para ejecución.

7.3. Ejecución de Pruebas en Contenedor

Ejecutar Pruebas Unitarias

```
docker run --rm monoschinos-mx mvn test
```

Ejecuta las 534 pruebas unitarias dentro de contenedor Docker aislado, verificando que aplicación funciona correctamente en ambiente contenedorizado sin dependencias del sistema operativo host. El flag -rm elimina automáticamente el contenedor después de ejecución liberando recursos. Resultado esperado indica Tests run: 534, Failures: 0, Errors: 0, Skipped: 0 confirmando que toda la funcionalidad implementada pasa validaciones automatizadas.

7.4. Ejecución de Aplicación GUI con Reenvío X11

La ejecución de aplicación JavaFX con interfaz gráfica desde contenedor Docker requiere configuración de reenvío X11 que permite que aplicación renderice ventanas en servidor X del host.

Paso 1: Verificar Variable DISPLAY

```
echo $DISPLAY
```

Verifica que variable de entorno DISPLAY está configurada (usualmente :0 o :1), indicando socket donde servidor X escucha conexiones de clientes gráficos.

Paso 2: Habilitar Reenvío X11

```
xhost +local:docker
```

Autoriza conexiones al servidor X desde contenedores Docker locales, permitiendo que aplicaciones contenedorizadas creen ventanas en display del host. Esta configuración debe ejecutarse únicamente una vez por sesión de usuario.

Paso 3: Ejecutar Contenedor con Reenvío X11

```
docker run -it --rm \
  --env="DISPLAY=$DISPLAY" \
  --volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" \
  --volume="$HOME/.Xauthority:$HOME/.Xauthority:rw" \
  --name=monoschinos-mx-app \
  monoschinos-mx
```

Ejecuta contenedor con configuración que:

- -it: Habilita modo interactivo con terminal
- -rm: Elimina contenedor automáticamente al finalizar
- -env="DISPLAY=\$DISPLAY": Propaga variable DISPLAY al contenedor
- -volume="/tmp/.X11-unix:...": Monta socket X11 del host en contenedor
- -volume="\$HOME/.Xauthority:...": Comparte credenciales de autenticación X11
- -name=monoschinos-mx-app: Asigna nombre descriptivo al contenedor

7.5. Método Alternativo de Reenvío X11

Si el método anterior no funciona debido a configuraciones específicas de distribución Linux o políticas de seguridad del servidor X, utilizar método alternativo:

```
docker run -it --rm \
  --env="DISPLAY" \
  --env="QT_X11_NO_MITSHM=1" \
  --volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" \
  --volume="/dev/dri:/dev/dri" \
  --device=/dev/dri \
  --name=monoschinos-mx-app \
  monoschinos-mx
```

Esta configuración adicional monta dispositivos DRI (Direct Rendering Infrastructure) permitiendo aceleración por hardware de gráficos, y deshabilita extensión MIT-SHM que puede causar conflictos en algunos ambientes.

7.6. Arquitectura del Contenedor

El contenedor Docker se construye sobre imagen base `openjdk:17-slim` que proporciona Java Runtime Environment 17 optimizado con tamaño reducido eliminando componentes innecesarios. La imagen incluye Maven 3.9+ instalado para automatización de compilación, todas las dependencias especificadas en `pom.xml` descargadas y disponibles en directorio `/app/target/lib`, y configuración de directorio `/.monoschinos` para persistencia de base de datos SQLite que almacena historial de tickets generados.

El comando de inicio por defecto ejecuta `mvn javafx:run` que lanza aplicación JavaFX con interfaz gráfica completa, pero puede sobrescribirse pasando comando alternativo al final de `docker run` para ejecutar pruebas, consola, o generar documentación según necesidades específicas del usuario.

Estado de la Implementación

8.1. Completitud del Proyecto

El proyecto se encuentra **100 % implementado** con todas las funcionalidades requeridas operativas y verificadas mediante suite exhaustiva de pruebas automatizadas. El sistema proporciona dos interfaces de usuario completamente funcionales (consola y JavaFX), gestión completa de inventario con componentes Intel y AMD, construcción de equipos personalizados y prearmados mediante builders especializados, verificación automática de compatibilidad con aplicación de adaptadores cuando sea necesario, instalación opcional de software mediante decoradores encadenables, generación de tickets detallados que documentan configuración completa del equipo, y persistencia de historial en base de datos SQLite que permite consultas de pedidos previos.

8.2. Suite de Pruebas Unitarias

El proyecto incluye **534 pruebas unitarias** desarrolladas con JUnit 5 que verifican exhaustivamente comportamiento de todos los componentes del sistema mediante validación automatizada de pre y post-condiciones esperadas. Las pruebas cubren componentes individuales verificando correcta inicialización de atributos y comportamiento de métodos de consulta, builders probando construcción paso a paso y delegación a director, estrategias de compatibilidad validando detección de conflictos y aplicación de adaptadores, decoradores verificando adición dinámica de software y cálculo acumulado de precios, fachada probando coordinación de operaciones complejas entre subsistemas, y persistencia validando almacenamiento y recuperación correcta de tickets desde base de datos SQLite.

La ejecución completa de la suite mediante `mvn test` produce resultado exitoso con **0 fallas, 0 errores y 0 pruebas omitidas**, confirmando que implementación satisface todos los requisitos funcionales especificados y mantiene calidad de código mediante validación continua durante desarrollo. Las pruebas utilizan fixtures para configurar estado inicial consistente, mocks para aislar componentes bajo prueba de dependencias externas, y assertions descriptivos que facilitan diagnóstico rápido cuando comportamiento observado difiere del esperado.

8.3. Interfaz Gráfica JavaFX

La aplicación JavaFX proporciona experiencia de usuario completa mediante ventana principal con navegación por paneles especializados que incluyen: panel de componentes con listas organizadas por tipo (CPU, RAM, MotherBoard, etc.) permitiendo selección individual mediante clics, panel de PC prearmadas con visualización de configuraciones predefinidas (gama baja, media, alta) mostrando componentes incluidos y precio total, panel de software con checkboxes para selección de aplicaciones opcionales (Windows, Office, Photoshop, AutoCAD, WSL) con validación que previene duplicados, panel de ticket que muestra resumen completo de configuración seleccionada con lista detallada de componentes, precio total calculado incluyendo software, y advertencias de compatibilidad cuando corresponda, y panel de historial que lista tickets previamente generados almacenados en base de datos SQLite permitiendo consulta de pedidos históricos.

La interfaz utiliza layouts responsivos que adaptan posicionamiento de componentes cuando usuario redimensiona ventana, estilos CSS personalizados que proporcionan apariencia visual profesional consistente con identidad de MonosChinos MX, iconos FontAwesome vectoriales que mejoran usabilidad mediante señales visuales claras de funcionalidad, y controles avanzados de ControlsFX como notificaciones toast que confirman operaciones exitosas y diálogos modales para confirmación de acciones destructivas.

8.4. Persistencia de Datos

El sistema implementa persistencia de tickets en base de datos SQLite embebida que almacena historial completo de equipos configurados por usuarios, incluyendo lista serializada de componentes en formato JSON mediante Gson, software instalado, precio total, fecha de generación, y advertencias de compatibilidad detectadas. La base de datos se crea automáticamente en directorio `/ .monoschinos` dentro del contenedor Docker, con migración de esquema ejecutada durante primera ejecución que configura tablas necesarias con índices apropiados para consultas eficientes.

El servicio de persistencia proporciona métodos para insertar nuevos tickets ejecutando prepared statements parametrizados que previenen inyección SQL, recuperar listado completo de tickets históricos con opción de filtrado por rango de fechas o precio, y consultar ticket específico por ID único asignado automáticamente. La implementación utiliza try-with-resources para garantizar liberación apropiada de conexiones y recursos JDBC incluso cuando ocurren errores durante operaciones de base de datos.

Funcionamiento del Sistema

El sistema opera mediante flujo interactivo que guía usuarios desde selección inicial del tipo de equipo deseado hasta generación de ticket final que documenta configuración completa con precio total y advertencias de compatibilidad.

9.1. Selección de Tipo de Compra

Al iniciar aplicación, usuario encuentra menú principal con dos opciones fundamentales: construir equipo personalizado donde selecciona individualmente cada componente según preferencias específicas, o seleccionar equipo prearmado eligiendo entre configuraciones predefinidas optimizadas para diferentes segmentos de mercado (gama baja, media, alta). La elección determina qué builder será utilizado por el director durante proceso de construcción.

9.2. Construcción de Equipo Personalizado

Cuando usuario selecciona modalidad personalizada, sistema presenta secuencialmente paneles de selección para cada tipo de componente requerido comenzando por procesador donde puede elegir entre CPUs Intel o AMD del catálogo, continuando con memoria RAM permitiendo selección de múltiples módulos para configurar capacidad total deseada, placa madre mostrando opciones compatibles con socket del procesador previamente seleccionado, tarjeta gráfica con rango desde opciones económicas hasta high-end para gaming en 4K, almacenamiento permitiendo combinación de múltiples discos HDD y SSD para balance entre capacidad y rendimiento, fuente de alimentación con sugerencia de potencia

mínima calculada según componentes seleccionados, y finalmente gabinete compatible con factor de forma de placa madre elegida.

Durante selección, sistema valida continuamente que cada componente agregado sea compatible con elementos previamente seleccionados mediante delegación a estrategias de compatibilidad apropiadas. Cuando detecta conflictos como procesador AMD en placa madre Intel, sistema notifica usuario mediante diálogo explicando el problema y ofreciendo aplicar adaptador automáticamente para resolver incompatibilidad, o cancelar selección para elegir componente alternativo compatible.

9.3. Selección de Equipo Prearmado

En modalidad prearmada, usuario visualiza tres configuraciones predefinidas con información detallada de componentes incluidos, precio total y casos de uso recomendados. La configuración de gama baja incluye Intel Core i3, 8GB RAM, placa ASUS B760, disco SSD 500GB, GPU GTX 1660, fuente 500W y gabinete Yeyian, apropiada para navegación web, ofimática y gaming casual con precio aproximado de \$26,000. La gama media incorpora AMD Ryzen 5 7600X, 16GB RAM, placa ASUS TUF B760, SSD 1TB, RTX 3060, fuente 800W y gabinete NZXT, orientada a gaming en 1080p/1440p y creación de contenido con precio de \$48,000. La gama alta presenta Intel Core i9-13900K, 32GB RAM, placa ASUS ROG Z790, SSD 2TB, RTX 4090, fuente Corsair 1500W y gabinete NZXT, diseñada para gaming en 4K, renderizado profesional y cargas de trabajo intensivas con precio de \$155,000.

Usuario selecciona configuración deseada mediante clic, cargando automáticamente todos los componentes especificados sin necesidad de selección individual. El sistema permite posteriormente modificar configuración prearmada intercambiando componentes específicos si usuario desea personalizar parcialmente, manteniendo ventaja de punto de partida optimizado sin configurar todo desde cero.

9.4. Personalización de Software

Después de configurar componentes de hardware, sistema presenta panel de software opcional donde usuario selecciona aplicaciones deseadas mediante checkboxes que incluyen Windows 10/11 con precio de \$2,500 proporcionando sistema operativo completo preinstalado, Microsoft Office 365 por \$1,800 con suite completa de productividad, Adobe Photoshop por \$2,200 para edición profesional de imágenes, AutoCAD por \$3,500 para diseño asistido por computadora, y terminal WSL por \$500 configurando Windows Subsystem for Linux con distribuciones preinstaladas.

El sistema valida que cada software sea seleccionado únicamente una vez mediante invocación a `tieneSoftware()` que recorre recursivamente cadena de decoradores detectando duplicados antes de agregar nuevo decorador. Cada software seleccionado incrementa precio total calculado automáticamente mediante suma recursiva de precios de decoradores encadenados.

9.5. Generación de Ticket

Al completar configuración, sistema genera ticket detallado que documenta: lista completa de componentes con nombre, marca y precio individual de cada elemento, software instalado con precios correspondientes, precio total del equipo calculado sumando todos los componentes y software, advertencias de compatibilidad indicando si se aplicaron adaptadores o existen recomendaciones para optimizar configuración, y fecha de generación con número de ticket único asignado automáticamente.

El ticket es presentado visualmente en panel dedicado permitiendo revisión completa antes de confirmar, y posteriormente almacenado en base de datos SQLite para consultas futuras del historial. Usuario puede imprimir o exportar ticket en formato PDF (funcionalidad adicional implementada mediante biblioteca iText) para documentación física del pedido.

9.6. Consulta de Historial

Panel de historial permite visualizar todos los tickets previamente generados ordenados cronológicamente con información resumida de fecha, precio total y número de componentes. Usuario puede seleccionar ticket específico para visualizar detalles completos recuperados desde base de datos, facilitando consulta

de configuraciones previas para referencia futura o duplicación de pedidos similares mediante funcionalidad de clonar configuración" que carga componentes de ticket histórico como punto de partida para nuevo pedido.

Diagramas del Sistema

El proyecto incluye tres diagramas fundamentales que documentan arquitectura, comportamiento e interacciones del sistema desde diferentes perspectivas complementarias.

10.1. Diagrama de Clases

El archivo `DiagramaClasesMonosChinoMX.drawio.png` presenta diagrama de clases completo del sistema mostrando todas las clases implementadas organizadas por paquetes, atributos y métodos de cada clase con visibilidad apropiada (public, private, protected), relaciones de herencia mediante flechas con triángulo vacío apuntando a superclase, relaciones de implementación de interfaces mediante líneas punteadas, asociaciones entre clases con multiplicidades que indican cardinalidad de relaciones, composiciones indicando relaciones parte-todo donde componente no puede existir independientemente del contenedor, y agregaciones representando relaciones más débiles donde componente mantiene existencia independiente.

El diagrama permite comprender arquitectura completa del sistema identificando patrones de diseño implementados mediante análisis de jerarquías de herencia y relaciones de composición características de cada patrón. La organización clara por paquetes facilita navegación identificando responsabilidades de cada subsistema y dependencias entre módulos.

10.2. Diagrama de Casos de Uso

El archivo `DiagramaUsosMonos.png` documenta casos de uso principales del sistema desde perspectiva del actor usuario, mostrando funcionalidades disponibles como construir equipo personalizado, seleccionar equipo prearmado, agregar software opcional, verificar compatibilidad, generar ticket, consultar historial y exportar configuración. El diagrama incluye relaciones de extensión (extend) que indican funcionalidades opcionales ejecutadas condicionalmente, e inclusiones (include) que representan subfuncionalidades requeridas obligatoriamente por casos de uso principales.

Este diagrama proporciona visión de alto nivel de qué puede hacer el sistema desde perspectiva del usuario final, facilitando validación de que requisitos funcionales especificados están completamente cubiertos por casos de uso implementados.

10.3. Diagrama de Secuencia

El archivo `DiagramaSecuencias.drawio.png` documenta mediante tres casos de uso fundamentales las interacciones temporales entre objetos durante ejecución de los escenarios principales del sistema, mostrando cómo colaboran doce componentes arquitectónicos (Usuario, ControladorPrincipal, Vista, SistemaEnsamblajeFacade, DirectorPC, Builders especializados, Inventario, ComputadoraBase, SoftwareDecorator, EstrategiaCompatibilidad, Ticket y ServicioPersistencia) para completar operaciones complejas mediante secuencias coordinadas de mensajes.

Caso de Uso 1: Construir PC Personalizada

Este escenario documenta el flujo completo de construcción interactiva donde el usuario selecciona individualmente cada componente. La secuencia inicia cuando Usuario invoca `construirPCPersonalizada()` en `ControladorPrincipal`, quien delega a `Vista` la presentación de interfaz de selección. `Vista` invoca `construirPCPersonalizada()` en `SistemaEnsamblajeFacade` que coordina la operación completa: instancia `PCPersonalizadoBuilder`, configura `DirectorPC` con el builder, e invoca construcción paso a paso mediante `construirPCCompleta()`.

El `DirectorPC` orquesta la secuencia de construcción invocando métodos especializados del builder

(construirCPU(), construirRAM(), construirMotherBoard(), construirGPU(), construirAlmacenamiento(), construirFuente(), construirGabinete()), cada uno consultando Inventario mediante obtenerComponentesPorTipo() para presentar opciones disponibles y agregando componente seleccionado a ComputadoraBase mediante agregarComponente().

Completada la selección de hardware, Vista consulta ¿agregarSoftware?() al Usuario, y en caso afirmativo invoca secuencialmente decoradores (WindowsDecorator, OfficeDecorator, etc.) verificando previamente mediante tieneSoftware() que cada aplicación no haya sido instalada duplicadamente. La computadora decorada es enviada a CompatibilidadMixta mediante verificarCompatibilidad() que retorna ResultadoCompatibilidad indicando si se requieren adaptaciones.

Finalmente, SistemaEnsamblajeFacade invoca generarTicket() recibiendo objeto Ticket que encapsula configuración completa, y lo persiste mediante guardarTicket() en ServicioPersistencia que almacena información serializada en base de datos SQLite. El ticket generado es retornado a Vista para presentación al Usuario.

Caso de Uso 2: Construir PC Prearmada

Este escenario simplifica la construcción mediante configuraciones predefinidas. Usuario invoca construirPCPrearmada() en ControladorPrincipal, quien delega a Vista la presentación de configuraciones disponibles (gama baja, media, alta) obtenidas mediante obtenerConfiguracionesPrearmadas() del Inventario.

Usuario selecciona configuración deseada, y Vista invoca construirPCPrearmada(gama) en SistemaEnsamblajeFacade que instancia PCPrearmadoBuilder configurado con la gama seleccionada. DirectorPC ejecuta construirPCCompleta() invocando secuencialmente métodos del builder que recuperan automáticamente componentes predefinidos del Inventario sin intervención del usuario, agregando cada elemento a ComputadoraBase.

El flujo continúa idénticamente al caso personalizado: Vista consulta sobre software opcional aplicando decoradores cuando corresponda, verifica compatibilidad mediante estrategia apropiada, genera ticket con configuración resultante, persiste en base de datos y presenta resultado final al Usuario.

Caso de Uso 3: Ver Historial de Compras

Este escenario permite consulta de tickets previamente generados. Usuario invoca verHistorial() en ControladorPrincipal, quien delega a Vista la presentación de interfaz de historial. Vista invoca obtenerHistorial() en ServicioPersistencia que consulta base de datos SQLite recuperando lista de tickets almacenados con información resumida (ID, fecha, precio total, número de componentes).

Vista presenta listado al Usuario permitiendo selección de ticket específico para visualización detallada, facilitando consulta de configuraciones históricas y referencia para pedidos futuros similares.

Notación del Diagrama

El diagrama utiliza convenciones estándar UML para diagramas de secuencia: líneas de vida verticales punteadas representan existencia temporal de objetos durante escenario, flechas horizontales sólidas indican invocaciones sincrónicas de métodos que esperan respuesta antes de continuar, flechas punteadas representan retornos de llamadas con valores de resultado, y rectángulos de activación en líneas de vida muestran períodos donde objeto está ejecutando operaciones. La organización clara por casos de uso con código de color (azul para personalizada, naranja para prearmada, rosa para historial) facilita comprensión independiente de cada flujo.

Este diagrama permite comprender flujo de control completo durante operaciones complejas identificando cuándo se crean objetos, qué mensajes son enviados en qué orden, y cómo se coordinan múltiples componentes distribuidos para completar funcionalidad de alto nivel mediante colaboración orquestada por la fachada que simplifica complejidad arquitectónica interna.

Notas Finales

El proyecto demuestra aplicación práctica de múltiples patrones de diseño trabajando coordinadamente para resolver problemática real de ensamblaje de computadoras personalizadas, cumpliendo requisitos funcionales y no funcionales mediante arquitectura bien estructurada que adhiere a principios SOLID maximizando mantenibilidad, extensibilidad y testabilidad del sistema desarrollado. La implementación completa con 534 pruebas unitarias exitosas, interfaz gráfica funcional, persistencia de datos operativa y despliegue mediante Docker garantiza calidad del producto entregado y satisfacción de especificaciones establecidas por MonosChinos MX para soporte de su expansión comercial exitosa.